

VICO: Visual-Inertial Continuous-Time Odometry Based on Generalized Hermite Spline

1st Haoyu Qi

School of Automation
Beijing Institute of Technology
Beijing, China
email: qiqiqihy@qq.com

2nd Zhen Li*

School of Automation
Beijing Institute of Technology
Beijing, China
email: zhenli@bit.edu.cn

3rd Haikuo Liu

School of Automation
Beijing Institute of Technology
Beijing, China
email: foreverlhk@bit.edu.cn

4th Xiangdong Liu

School of Automation
Beijing Institute of Technology
Beijing, China
email: xdliu@bit.edu.cn

5th Fang Deng

School of Automation
Beijing Institute of Technology
Beijing, China
email: dengfang@bit.edu.cn

Abstract—Continuous-time simultaneous localization and mapping (SLAM) facilitates the seamless fusion of asynchronous and high update-rate sensors. The traditional continuous-time parameterization adopts the cumulative B-splines, causing the complexity in implementation and abstraction of control points. To address these issues, this paper proposes a continuous-time visual-inertial odometry (VIO) based on generalized Hermite spline. The analytical temporal derivatives and Jacobians with respect to the control points are derived, so that the VIO is further formulated as a sliding window based optimization. To validate the efficacy of the proposed method, the extensive evaluations are conducted on the TUM VI and EuRoC dataset. The results demonstrate the state-of-the-art accuracy and real-time performance. Our implementation is fully open-source at <https://github.com/FALCONS-Lab/VICO>.

Index Terms—Continuous-time SLAM, visual-inertial odometry, sensor fusion.

I. INTRODUCTION

Simultaneous localization and mapping (SLAM) facilitates autonomous navigation by providing the accurate and robust pose estimation with seamless fusion of various sensors [1]. Due to the low cost and the complementary benefits in observability and update rate, the visual-inertial odometry (VIO), which composes of camera and inertial measurement unit (IMU), becomes a commercially mainstream localization approach. SLAM is in nature considered a state estimation problem, and its feasible solutions are categorized into filtering-based and optimization-based methods. For optimization-based SLAM, 6-degree-of-freedom (6-DoF) poses at the sampling instants of sensors are conventionally taken as state variables, and the corresponding approaches employing this parameterization are referred to as discrete-time SLAM. However, such parameterization can be challenged in real-time performance, especially when SLAM system deals with high update-rate IMUs or heterogeneous sampling rates between cameras and IMUs. Various strategies have been proposed to improve the efficiency of SLAM optimization,

e.g., the adoption of sliding windows to limit the scale of optimization [2], and the utilization of IMU pre-integration to unify the rate of visual and inertial measurements [3].

Compared to discrete-time SLAM above, continuous-time SLAM (CT-SLAM) employs the continuous-time parameterization, where ego-motion is modeled as a mapping or stochastic process from time to 6-DoF poses determined by a series of discrete control points. Such parameterization has been proven to effectively reduce the state space dimensionality, and its explicit consideration on time allows the analytical extraction of higher-order kinematic quantities (e.g. linear and angular velocities, accelerations) without introducing additional state variables. Overall, the continuous-time parameterization approaches can be classified into interpolation [4], [5], spline [6]–[9], hierarchical wavelets [10], and Gaussian processes [11], [12]. Although most practical implementations used the uniform cumulative B-splines to represent the continuous-time trajectories, there are two main challenges affecting the overall performance, i.e., 1) *Structural complexity*: in cumulative B-spline, multiple control points on the Lie group are utilized for the computation of each pose in state estimation, and conversely, each individual control point influences the determination of multiple poses, resulting in complicated Jacobian evaluation and a more dense optimization structure [13]; 2) *the abstraction of control points*: the fact that B-splines curves do not pass through their control points and the absence of a clear correspondence between each control points and kinematic quantities, creates difficulties in establishing suitable initial values for these control points, which impacts the convergence behavior of iterative optimization methods sensitive to initial guess.

To deal with these problems, we utilize generalized Hermite spline as continuous-time parameterization [14]. As an interpolation spline curve, the generalized Hermite spline not only provides the enough continuity, but also renders the control points proportional to kinematic quantities, which enable the

efficient initialization of control points and effectively reduces the complexity of optimization structure. In this paper, we further propose a visual-inertial continuous-time odometry based on generalized Hermite spline, *i.e.*, VICO. Therefore, the contributions of this work can be summarized as follows.

- 1) The analytical Jacobian of generalized Hermite spline with regard to its control points is derived, and a monocular visual-inertial continuous-time odometry (VICO) is proposed with the sliding window optimization and marginalization strategies.
- 2) The proposed VICO system is evaluated on real-world datasets, demonstrating its state-of-the-art accuracy and real-time performance. The implementation of our system is fully open-source.

II. RELATED WORK

The key of CT-SLAM lies in the continuous-time parameterization to accurately model the temporal characteristics of various sensors. Sensors such as rolling shutter cameras, event cameras, or light detection and ranging (LiDAR) devices, etc., do not complete the sampling instantaneously, so that one single sampling may correspond to multiple poses at different timestamps. The continuous-time parameterization can represent these poses with as few states as possible. Lovegrove *et al.* modeled the rolling shutter effect in VIO using the cumulative uniform B-spline on $SE(3)$, and trivially synthesized the accelerometer and gyroscope measurements [9]. Lang *et al.* considered the rolling shutter effect and proposed Ctrl-VIO with the line delay online calibration [15]. Mueggler *et al.* proposed a VIO with the event camera by integrating asynchronous events and high update-rate IMU measurements in a continuous-time framework [16]. In the proposed SLICT system [5], Nguyen *et al.* designed a continuous-time maximum-a-posteriori optimization that directly utilized the raw LiDAR data instead of deskewed points. General multi-sensor SLAM systems typically updates the measurements at different timestamps and rates of sensors. Towards this issue, the continuous-time parameterization can enable the fusion of multi-sensor data without hardware synchronization. Hug *et al.* proposed a continuous-time stereo VIO robust to stereo streams that the left and right images were captured at different rates [7]. Additionally, the tight coupling of asynchronous LiDAR, IMU and camera data via continuous-time parameterization was investigated in [6] and [13].

Among various continuous-time parameterization methods, cumulative B-spline won the most widespread usage, whose utilization has gradually evolved from the initial joint form on $SE(3)$ to the split form on $\mathbb{R}^3$ and $SO(3)/S^3$ due to the latter's improved efficiency and convergence [17]. Although the analytical Jacobian of the split form with regard to its control points was provided in [18], which significantly sped up the optimization, the aforementioned complexities and abstractness in nature still persisted. To tackle these difficulties, a marginalization method was proposed in [15] specifically for cumulative B-spline in sliding window optimization by introducing a more efficient IMU pre-integration prior instead

of the raw measurements prior. In [13], the initialization of the abstract control points was achieved through an optimization including velocity and raw measurements factors.

On the other hand, many works were dedicated to adopting new continuous-time parameterization methods. Mo and Sattar formulated VIO as a constrained nonlinear optimization problem using cubic spline with boundary conditions [8]. Johnson *et al.* considered the differential flatness property of dynamic systems and defined B-spline in their flat output space to avoid time-consuming Jacobian evaluation on Lie groups [19].

Motivated by the simplification on initialization and estimation of control points, we utilize the generalized Hermite spline as continuous-time parameterization and demonstrate its applicability in VIO.

III. METHODOLOGY

We first introduce the reference frames and operations on Lie groups used in this paper. The world frame, body frame and camera frame are denoted by w , b and c , respectively, where the body frame is equal to the IMU frame. The world frame is considered fixed, with the gravity vector aligned downward along its z -axis. $\mathbf{p}^w \in \mathbb{R}^3$ represents a vector in frame w , and $\mathbf{q}_b^w \in S^3$ represents the rotation from frame b to frame w . $\mathbf{q}^*$ represents the conjugate quaternion of $\mathbf{q} \in S^3$. $\text{Exp}(\cdot)$ maps Lie algebra to Lie group, while $\text{Log}(\cdot)$ is its inverse operation. $\mathbf{J}_l(\cdot)$ and $\mathbf{J}_r(\cdot)$ are the left and right Jacobian on S^3 defined in [20], respectively.

A. Spline Parameterization

The generalized Hermite spline used in this paper consists of a position Hermite spline in $\mathbb{R}^3$ and a cumulative orientation Hermite spline on S^3 . To guarantee the continuity of the synthesized acceleration, the position Hermite spline $\mathbf{p}(u) : [0, 1] \rightarrow \mathbb{R}^3$ is formulated as a quintic polynomial defined by two end positions $\mathbf{p}_a, \mathbf{p}_b$, two end velocities $\mathbf{v}_a, \mathbf{v}_b$ and two end accelerations $\mathbf{a}_a, \mathbf{a}_b$. Note that a Hermite spline can be represented by a Bézier spline, so that the specific form of $\mathbf{p}(u)$ is given by

$$\mathbf{p}(u) = \sum_{k=0}^5 B_{k,5}(u) \mathbf{P}_k, \quad (1)$$

where $B_{k,l}(u)$ denotes the Bézier basis function defined as $B_{k,l}(u) = \binom{l}{k} u^k (1-u)^{l-k}$, and $\mathbf{P}_k$ is given by

$$\begin{cases} \mathbf{P}_0 = \mathbf{p}_a, \\ \mathbf{P}_1 = \mathbf{p}_a + 0.2\mathbf{v}_a, \\ \mathbf{P}_2 = \mathbf{p}_a + 0.4\mathbf{v}_a + 0.05\mathbf{a}_a, \\ \mathbf{P}_3 = \mathbf{p}_b - 0.4\mathbf{v}_b + 0.05\mathbf{a}_b, \\ \mathbf{P}_4 = \mathbf{p}_b - 0.2\mathbf{v}_b, \\ \mathbf{P}_5 = \mathbf{p}_b. \end{cases} \quad (2)$$

By this specific assignment, the above-defined position Hermite spline satisfies the following boundary properties,

$$\begin{cases} \mathbf{p}(0) = \mathbf{p}_a, \mathbf{p}(1) = \mathbf{p}_b, \\ \dot{\mathbf{p}}(0) = \mathbf{v}_a, \dot{\mathbf{p}}(1) = \mathbf{v}_b, \\ \ddot{\mathbf{p}}(0) = \mathbf{a}_a, \ddot{\mathbf{p}}(1) = \mathbf{a}_b. \end{cases} \quad (3)$$

On the other hand, to similarly guarantee the continuity of the synthesized angular velocity, the cumulative orientation Hermite spline $q(u) : [0, 1) \rightarrow S^3$ is formulated as the cubic cumulative Hermite spline defined in [14]. This spline is defined by two end unit quaternions q_a, q_b and two end angular velocities ω_a, ω_b . The specific form of $q(u)$ is given by

$$q(u) = q_0 \prod_{k=1}^3 \text{Exp} \left[\tilde{B}_{k,3}(u) \omega_k \right], \quad (4)$$

where $\tilde{B}_{k,l}(u)$ denotes the cumulative Bésier basis function. $\tilde{B}_{k,l} = \sum_{i=k}^l B_{i,l}(u)$, and $\omega_k = \text{Log}(q_{k-1}^* q_k)$ with

$$\begin{cases} q_0 = q_a, \\ q_1 = q_a \text{Exp}(\omega_a/3), \\ q_2 = q_b \text{Exp}(-\omega_b/3), \\ q_3 = q_b. \end{cases} \quad (5)$$

As shown in (4), the cumulative Hermite spline shares a similar generalization with the cumulative B-spline commonly used in CT-SLAM, so that the analytical form of its angular velocity $\omega(u)$ can be obtained by the same iterative scheme as in [18]. Besides, the cumulative orientation Hermite spline defined above satisfies the following boundary properties,

$$\begin{cases} q(0) = q_a, & q(1) = q_b, \\ \omega(0) = \omega_a, & \omega(1) = \omega_b. \end{cases} \quad (6)$$

In order to further obtain the complete Jacobian of the residuals via the chain rule, we first derive the Jacobian of ω_k ($k = 1, 2, 3$) with regard to q_a, q_b, ω_a and ω_b as follows.

$$\mathbf{J}_{q_a}^{\omega_1} = \mathbf{J}_{q_b}^{\omega_1} = \mathbf{J}_{\omega_a}^{\omega_1} = \mathbf{0}_{3 \times 3}, \quad \mathbf{J}_{\omega_b}^{\omega_1} = \mathbf{I}_{3 \times 3}, \quad (7)$$

$$\begin{aligned} \mathbf{J}_{q_a}^{\omega_2} &= -\mathbf{J}_l^{-1}(\omega_2) \mathbf{R} [\text{Exp}(-\omega_1)], \\ \mathbf{J}_{q_b}^{\omega_2} &= \mathbf{J}_r^{-1}(\omega_2) \mathbf{R} [\text{Exp}(\omega_3)], \\ \mathbf{J}_{\omega_a}^{\omega_2} &= -\frac{1}{3} \mathbf{J}_l^{-1}(\omega_2) \mathbf{J}_r(\omega_1), \\ \mathbf{J}_{\omega_b}^{\omega_2} &= -\frac{1}{3} \mathbf{J}_r^{-1}(\omega_2) \mathbf{J}_l(\omega_3), \end{aligned} \quad (8)$$

$$\mathbf{J}_{q_a}^{\omega_3} = \mathbf{J}_{q_b}^{\omega_3} = \mathbf{J}_{\omega_a}^{\omega_3} = \mathbf{0}_{3 \times 3}, \quad \mathbf{J}_{\omega_b}^{\omega_3} = \mathbf{I}_{3 \times 3}, \quad (9)$$

where $\mathbf{J}_b^a$ represents the Jacobian of a with regard to b , and $\mathbf{R}(q) \in SO(3)$ represents the same rotation as $q \in S^3$.

In brief, the generalized Hermite spline used in this paper is an interpolation spline, whose control points are the positions, velocities, accelerations, orientations and angular velocities of its endpoints.

Given a set of uniformly distributed temporal knots $t_0, t_1, t_2, \dots$, our continuous-time parameterization can be obtained by constructing the generalized Hermite spline between the adjacent knots as

$$p_i(u) = p_i[u(t)], \quad q_i(u) = q_i[u(t)], \quad (10)$$

$$u(t) = \frac{t - t_i}{\Delta t}, \quad t_i \leq t < t_{i+1}, \quad (11)$$

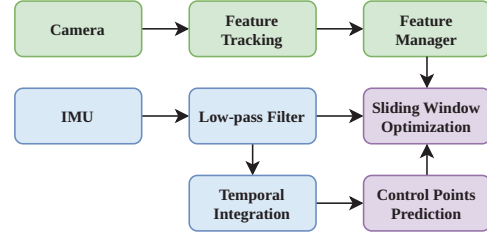


Fig. 1. Pipeline of VICO.

where $\Delta t = t_{i+1} - t_i$ is the constant time interval. Note that the boundary properties guarantee the continuity of acceleration and angular velocity throughout the entire time domain so as to smoothly synthesize the IMU measurements, which will be used to construct the inertial residual in Section III-D.

B. System Overview

Fig. 1 shows the pipeline structure of VICO. The images captured by the camera are transmitted to a visual front-end refer to [2], which tracks the features between adjacent images to establish the data association. In a new coming image, the features from the previous image are firstly tracked via the KLT optical flow [21], followed by the extraction of new Shi-Tomasi corners [22] to ensure enough features in each image. The associated features are lifted onto the normalized plane and added to the feature manager as landmarks. The keyframes are also selected in the front-end according to the similar criteria as in [2]. Meanwhile, the raw IMU measurements are first processed through a low-pass filter to mitigate the impact of outliers, after which the filtered measurements are integrated to predict the initial values for control points.

The sliding window optimization estimates the state vector by fully utilizing the landmark observations and filtered IMU measurements within the window, alongside the prior information from marginalization. The state vector in the sliding window is defined as

$$\begin{aligned} \chi &= [s_0, s_1, \dots, s_n, \lambda_0, \lambda_1 \dots \lambda_m], \\ s_i &= [p_{b_i}^w, v_{b_i}^w, a_{b_i}^w, q_{b_i}^w, \omega_{b_i}^w, b_i^{\text{acc}}, b_i^{\text{gyr}}], \end{aligned} \quad (12)$$

where n is the size of the sliding window, and s_i is the state at time instance t_i , composed of control points and acceleration/gyroscope biases $b_i^{\text{acc}}/b_i^{\text{gyr}}$. m is the number of landmarks within the window, and λ_j is the inverse depth of each landmark's first observation in camera frame.

C. Visual Factor

The continuous-time parameterization based on generalized Hermite spline enables the use of simpler discrete-time visual factor. Given the temporal homogeneity of the image stream, we align the control points with the image timestamps, which allows for the direct querying of the positions and orientations corresponding to each image. Therefore, the computation of the analytical Jacobian significantly benefits from this specific alignment. Considering the k -th landmark observed in both the i -th and j -th frames within the window, in which their

normalized coordinates in these frames are denoted as $\mathbf{n}_i^k$ and $\mathbf{n}_j^k$, respectively, the visual factor $\mathbf{r}_V(\chi, \mathbf{n}_i^k, \mathbf{n}_j^k)$ is given by

$$\mathbf{r}_V(\chi, \mathbf{n}_i^k, \mathbf{n}_j^k) = \begin{bmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \end{bmatrix} \frac{\mathbf{L}^{c_j}}{\mathbf{e}_3^\top \mathbf{L}^{c_j}} - \mathbf{n}_j^k, \quad (13)$$

$$\mathbf{L}^{c_j} = \mathbf{q}_{c_i}^{c_j} \frac{1}{\lambda_k} \begin{bmatrix} \mathbf{n}_i^k \\ 1 \end{bmatrix} + \mathbf{p}_{c_i}^{c_j},$$

$$\mathbf{q}_{c_i}^{c_j} = \mathbf{q}_b^c \mathbf{q}_w^{b_j} \mathbf{q}_{b_i}^w \mathbf{q}_c^b, \quad (14)$$

$$\mathbf{p}_{c_i}^{c_j} = \mathbf{q}_b^c \left[\mathbf{q}_w^{b_j} \left(\mathbf{q}_{b_i}^w \mathbf{p}_c^b + \mathbf{p}_{b_j}^w - \mathbf{p}_{b_i}^w \right) - \mathbf{p}_c^b \right],$$

where the constants $\mathbf{q}_b^c$ and $\mathbf{p}_c^b$ denote the pre-calibrated extrinsic parameters. $\mathbf{e}_i$ denotes the i -th column of the identity matrix $\mathbf{I} \in \mathbb{R}^{3 \times 3}$.

The analytical Jacobian of $\mathbf{r}_V(\chi, \mathbf{n}_i^k, \mathbf{n}_j^k)$ with regard to χ is trivial via the chain rule as

$$\mathbf{J}_{\chi}^{\mathbf{r}_V(\chi, \mathbf{n}_i^k, \mathbf{n}_j^k)} = \mathbf{J}_{\mathbf{L}^{c_j}}^{\mathbf{r}_V(\chi, \mathbf{n}_i^k, \mathbf{n}_j^k)} \mathbf{J}_{\chi}^{\mathbf{L}^{c_j}}. \quad (15)$$

D. Inertial Factor

Considering the filtered IMU measurements, i.e., acceleration $\hat{\mathbf{a}}^b$ and angular velocity $\hat{\omega}^b$, at timestamp $t \in [t_i, t_{i+1})$ between the i -th and $(i+1)$ -th frames within the window, the inertial factor $\mathbf{r}_I(\chi, \hat{\mathbf{a}}^b, \hat{\omega}^b)$ is given by

$$\mathbf{r}_I(\chi, \hat{\mathbf{a}}^b, \hat{\omega}^b) = \begin{bmatrix} \mathbf{r}_{\omega}(\chi, \hat{\omega}^b) \\ \mathbf{r}_{\mathbf{a}}(\chi, \hat{\mathbf{a}}^b) \\ \mathbf{b}_{i+1}^{\text{gyr}} - \mathbf{b}_i^{\text{gyr}} \\ \mathbf{b}_{i+1}^{\text{acc}} - \mathbf{b}_i^{\text{acc}} \end{bmatrix}, \quad (16)$$

$$\mathbf{r}_{\omega}(\chi, \hat{\omega}^b) = \omega_i(u) - \hat{\omega}^b + \mathbf{b}_i^{\text{gyr}},$$

$$\mathbf{r}_{\mathbf{a}}(\chi, \hat{\mathbf{a}}^b) = \mathbf{q}_i^*(u) (\mathbf{a}_i(u) + \mathbf{g}^w) - \hat{\mathbf{a}}^b + \mathbf{b}_i^{\text{acc}},$$

where $\omega_i(u)$ and $\mathbf{a}_i(u)$ denote the angular velocity and acceleration synthesized via our continuous-time parameterization, respectively. $\mathbf{g}^w$ is the gravity vector in frame w . The timestamp t is converted to $u \in [0, 1)$ by (11).

The analytical Jacobian of $\mathbf{r}_{\omega}(\chi, \hat{\omega}^b)$ and $\mathbf{r}_{\mathbf{a}}(\chi, \hat{\mathbf{a}}^b)$ with regard to χ is given by

$$\mathbf{J}_{\chi}^{\mathbf{r}_{\omega}(\chi, \hat{\omega}^b)} = \mathbf{J}_{\omega_i(u)}^{\mathbf{r}_{\omega}(\chi, \hat{\omega}^b)} \mathbf{J}_{\chi}^{\omega_i(u)}, \quad (17)$$

$$\mathbf{J}_{\chi}^{\omega_i(u)} = \sum_{k=1}^3 \mathbf{J}_{\omega_k}^{\omega_i(u)} \mathbf{J}_{\chi}^{\omega_k},$$

$$\mathbf{J}_{\chi}^{\mathbf{r}_{\mathbf{a}}(\chi, \hat{\mathbf{a}}^b)} = \mathbf{J}_{\mathbf{q}_i^*(u)}^{\mathbf{r}_{\mathbf{a}}(\chi, \hat{\mathbf{a}}^b)} \mathbf{J}_{\chi}^{\mathbf{q}_i^*(u)} + \mathbf{J}_{\mathbf{a}_i(u)}^{\mathbf{r}_{\mathbf{a}}(\chi, \hat{\mathbf{a}}^b)} \mathbf{J}_{\chi}^{\mathbf{a}_i(u)}, \quad (18)$$

$$\mathbf{J}_{\chi}^{\mathbf{q}_i^*(u)} = \mathbf{J}_{\mathbf{q}_0}^{\mathbf{q}_i^*(u)} \mathbf{J}_{\chi}^{\mathbf{q}_0} + \sum_{k=1}^3 \mathbf{J}_{\omega_k}^{\mathbf{q}_i^*(u)} \mathbf{J}_{\chi}^{\omega_k},$$

where the part marked with wavy underline is obtained from (7)–(9), while the rest part is consistent with the derivation in [18].

E. Sliding Window Optimization with Marginalization

The architecture of sliding window optimization and corresponding factor graph are shown in Fig. 2(a) and Fig. 2(b), respectively. Given the state vector in (12), the sliding window optimization can be formulated as

$$\min_{\chi} \left\{ \|\mathbf{r}_P(\chi)\|^2 + \sum_{(i,j) \in \mathcal{L}_k} \rho \left(\|\mathbf{r}_V(\chi, \mathbf{n}_i^k, \mathbf{n}_j^k)\|_{\Sigma_C}^2 \right) + \sum_{l \in \mathcal{B}} \left\| \mathbf{r}_I(\chi, \hat{\mathbf{a}}^{b_l}, \hat{\omega}^{b_l}) \right\|_{\Sigma_I}^2 \right\}, \quad (19)$$

where $\|\cdot\|$ denote the Euclidean distance, and $\|\cdot\|_{\Sigma}$ denotes the Mahalanobis distance defined by covariance matrix Σ . $\mathbf{r}_P(\chi)$ is the prior factor obtained from the previous optimization. $\mathcal{L}_k$ is a set of observation of landmark k , where each element (i, j) represents the first and any other observations of landmark k . $\mathcal{B}$ is the set of all IMU measurements between consecutive frames within the window. $\rho(\cdot)$ denotes the Huber loss. Due to the analytical Jacobian of all factors derived, the optimization problem (19) can be efficiently solved by Ceres Solver [23].

To launch the optimization, a discrete-time initialization similar to [2] is adopted, which aligns the visual observations with the IMU pre-integration to solve for the initial coordinate frame, scale and gyroscope bias. Subsequently, all IMU pre-integration information is marginalized for the seamless switching to our continuous-time optimization.

Based on the distribution of keyframes within the sliding window, two marginalization strategies are developed specifically. First, when the second new frame is a keyframe, we marginalize the oldest frame as depicted in Fig. 2(c). All its related factors are then turned into a prior factor via the Schur complement, and the inverse depth of all its observed landmarks are transferred to the second old frame. On the contrary, when the second new frame is not a keyframe, we marginalize it as depicted in Fig. 2(d). Only its related inertial and prior factors are turned into a new prior factor, while all its visual observations are directly dropped, by means of which avoids introducing a large number of inverse depth states into prior factor.

IV. EXPERIMENTS

The proposed VICO system is evaluated on two real-world datasets, i.e., TUM VI [24] and EuRoC [25]. The system is further compared against a monocular continuous-time VIO referred to as SplineVIO [8], which also adopted continuous-time parameterization as an alternative to IMU pre-integration. However, SplineVIO simply used the conventional spline as continuous-time parameterization and formulated VIO as a constrained nonlinear optimization problem. Given that the front-end in our system is based on VINS-Mono [2], we also evaluate it as a reference to demonstrate the comparison between discrete-time and continuous-time VIO methods.

In our experiments, the extrinsic parameters between camera and IMU are considered pre-calibrated, and both the relocalization and pose graph optimization are disabled in VINS-Mono to guarantee a pure VIO evaluation. We take the root

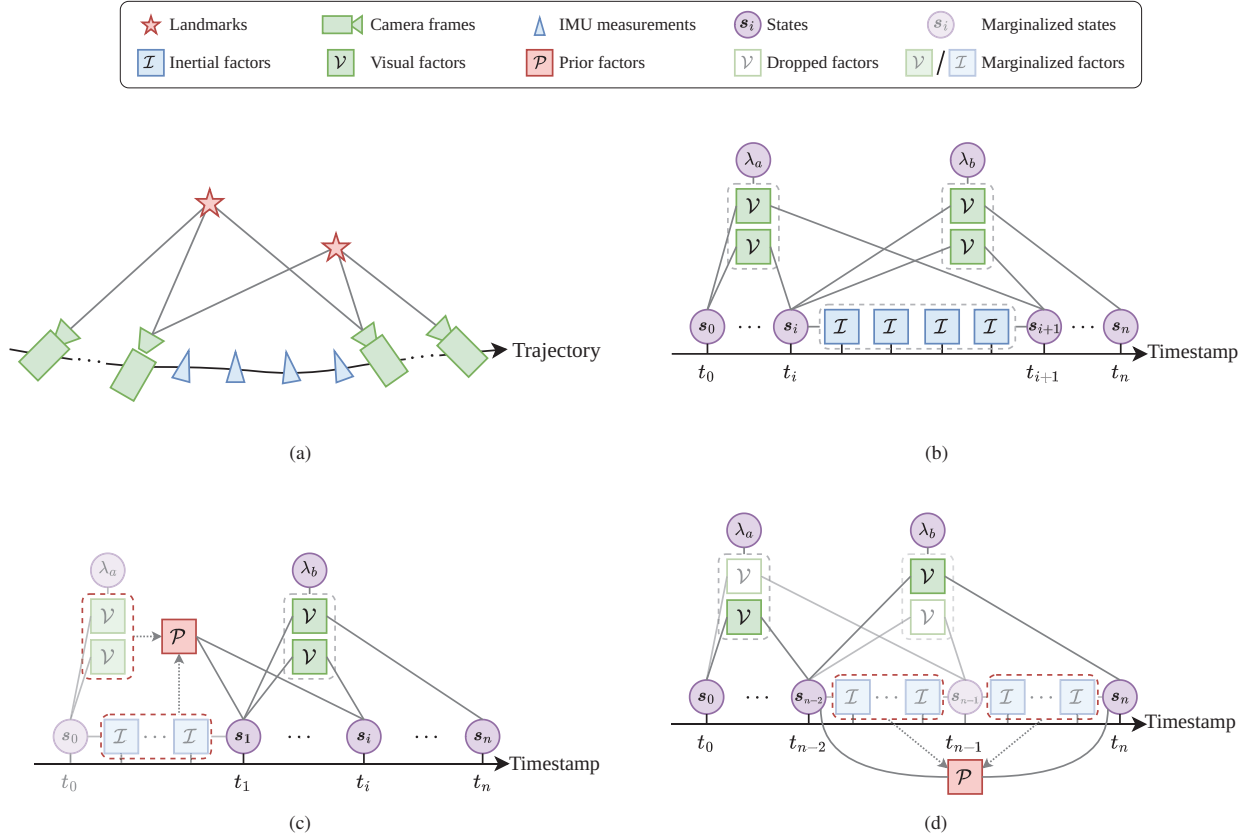


Fig. 2. Illustration of the sliding window optimization. (a) The asynchronous visual and inertial measurements within the window. (b) A factor graph corresponding to (a). (c) The marginalization strategy when the second new frame is a keyframe. The oldest frame is marginalized, all its related factors are turned into a prior factor. (d) The marginalization strategy when the second new frame is not a keyframe. The frame itself is marginalized, its visual factors are directly dropped, while other factors are turned into a prior factor.

mean square error (RMSE) of absolute pose error (APE) as the metric for the accuracy evaluation, which can be computed by toolbox [26]. The estimated trajectory is aligned with the ground truth via $SE(3)$ transformation. Each VIO method is executed for three times on each datasets so as to take the average.

A. TUM VI Dataset

The TUM VI dataset is a visual-inertial dataset collected by handheld devices, which provides the image in 20Hz from stereo fisheye cameras alongside 200Hz IMU measurements. All sensors are hardware-synchronized and calibrated. The “room” sequences within this dataset are selected for evaluation as there has the ground truth captured by a motion capture system available for the entire trajectory. We use the images from the left camera as visual input.

The RMSE of APE is shown in TABLE I for the proposed VICO, SplineVIO and VINS-Mono. Compare to SplineVIO, our VICO method achieves lower RMSE on most sequences. Specifically, SplineVIO suffers from visual tracking loss at 103s within the last 10s of the “room4” sequence and at 76s of the “room5” sequence, which prevents it from completing

the entire localization. However, there has no tracking loss occurred in our VICO method, indicating its superior robustness. Compared with VINS-Mono representing for discrete-time approach, the continuous-time approaches show less advantage on accuracy in confined scenarios because the trajectories in these scenarios often present significant maneuvering, especially for handheld devices, which cause the high volatility in IMU measurements. As a result, the spline-based continuous-time parameterizations cannot sustain the efficacious state estimation in such scenarios, struggling to accurately reflect the true dynamic behavior. On the contrary, the use of pre-integration can effectively mitigate these volatility through integration.

B. EuRoC Dataset

The EuRoC dataset is collected by a hex-rotor helicopter equipped with a visual-inertial sensor unit, which provides the images in 20Hz from stereo camera in front-down looking position alongside 200Hz IMU measurements. All sensors are also hardware-synchronized and calibrated. The dataset contains two different scenarios, i.e., “Machine Hall” (MH) and “Vicon Room” (V), each of which is accompanied with

TABLE I
THE RMSE(m) OF APE IN TUM VI DATASET. THE BEST RESULTS ARE IN BOLD AND THE SECOND BEST RESULTS ARE UNDERLINED.

Sequence	room1	room2	room3	room4	room5	room6
SplineVIO	0.142	0.235	0.191	0.215	×	<u>0.128</u>
VICO	<u>0.136</u>	<u>0.183</u>	<u>0.226</u>	<u>0.180</u>	0.176	0.166
VINS-Mono	0.085	0.109	0.394	0.044	<u>0.204</u>	0.115

TABLE II
THE RMSE(m) OF APE IN EUROC-MH DATASET. THE BEST RESULTS ARE IN BOLD AND THE SECOND BEST RESULTS ARE UNDERLINED.

Sequence	MH01	MH02	MH03	MH04	MH05
SplineVIO	0.140	0.113	0.256	1.653	0.194
VICO	0.357	<u>0.135</u>	0.122	0.303	0.430
VINS-Mono	<u>0.192</u>	0.167	<u>0.234</u>	<u>0.347</u>	<u>0.319</u>

TABLE III
THE RMSE(M) OF APE IN EUROC-V DATASET. THE BEST RESULTS ARE IN BOLD AND THE SECOND BEST RESULTS ARE UNDERLINED.

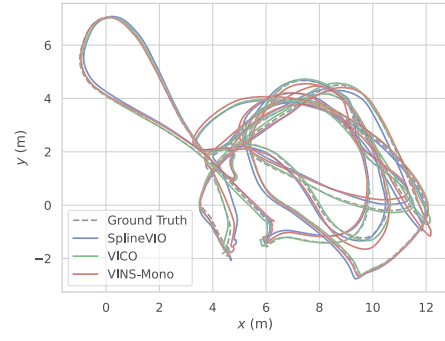
Sequence	V101	V102	V103	V201	V202	V203
SplineVIO	0.154	<u>0.439</u>	×	0.094	0.222	×
VICO	<u>0.125</u>	×	×	0.087	<u>0.148</u>	×
VINS-Mono	0.079	0.098	0.147	<u>0.088</u>	0.123	0.304

full ground truth obtained via a laser tracker and a Vicon motion capture system, respectively. Similar to the evaluation on TUM VI dataset, we use the images from the left camera as visual input.

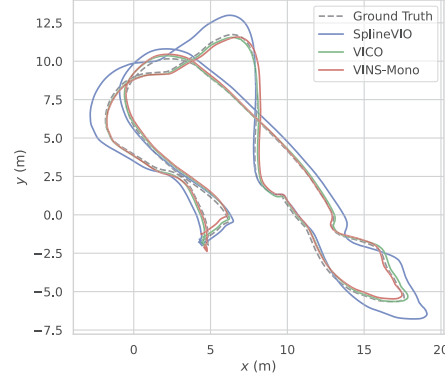
The RMSE of APE is shown in TABLE II and III. In the EuRoC dataset, the performance of the proposed VICO system is comparable to that of SplineVIO. In the “Machine Hall” sequences, our method performs best on ‘MH03’ and ‘MH04’ sequences, as shown in detailed in Fig. 3, and performs moderately on the remaining sequences. In the “Vicon Room” sequences, both our method and SplineVIO face challenges to accomplish the localization under conditions of fast motion and motion blur. However, in the sequences, where both work, our method still outperforms SplineVIO in accuracy. Compared to the discrete-time counterpart, the continuous-time method achieves better results in spacious scenarios, but performs worse in confined scenarios, which is consistent with the conclusion on the TUM VI dataset. We hold the opinion that the main reason for this case is the insufficient flexibility of continuous-time parameterization to represent ego-motion, making it difficult to adaptively deform based on the motion reflected by sensors, such as varying the frequency of control point insertion in response to the intensity of ego-motion. In the future research, we will consider the non-uniform splines to address this limitation and enhance adaptability.

C. Efficiency Analysis

Considering the similarity in structure between the proposed VICO and VINS-Mono, we compare the runtime between



(a) MH03



(b) MH04

Fig. 3. Estimated trajectory comparison of SplineVIO, VICO and VINS-Mono in EuRoC dataset.

TABLE IV
THE RUNTIME STATISTICS ON MH03 IN EUROC DATASET.
DATA FORMAT: MEAN(STANDARD DEVIATION).

	VICO	VINS-Mono	SplineVIO
Frontend (ms)	11.81(3.47)	6.56(6.03)	—
Optimization (ms)	13.97(4.35)	12.45(2.12)	30.66(6.18)
Marginalization (ms)	3.68(1.81)	3.11(1.83)	9.15(1.66)

these two methods on “MH03” sequence in EuRoC dataset, whose results are shown in TABLE IV. Although SplineVIO is an implementation based on DSO [27], employing a direct method for visual data association that differs from VICO and VINS-Mono, we also record its runtime as a reference for continuous-time methods. The experiment runs with an Intel i7-12700H CPU at 2.7GHz and 16GB RAM.

The front-ends of both VICO and VINS-Mono run on dedicated threads. Our front-end has similar process to VINS-Mono, and the discrepancies in runtime is due to VINS-Mono’s frequency control for the publication of front-end results. Not every incoming frame undergoes a complete feature detection process, which accounts for its lower mean runtime and the higher standard deviation. For SplineVIO, the use of direct method allows it to directly construct photometric

errors in the back-end from the original image instead of reprojection errors, so that eliminate the necessity for a distinct thread dedicated to feature extraction and matching.

Owing to the complexity of continuous-time parameterization, our method needs to calculate more residual terms in optimization and marginalization because each individual IMU measurement in our method is transformed into a residual term. However, for VINS-Mono, all IMU measurements between two keyframes are pre-integrated together into a joint residual term. Both being continuous-time parameterizations, our method outperforms SplineVIO in both optimization and marginalization runtime, mainly due to two reasons: firstly, the use of generalized Hermite spline introduce no additional constraints into the optimization, simplifying the structure complexity of the optimization problem; secondly, under the same IMU residuals, our visual reprojection residuals entail lower computational loads compared to the photometric residuals utilized by SplineVIO. In conclusion, although the complexity of continuous-time parameterization leads to longer runtime for optimization and marginalization, our method still can sustain the real-time performance and the mean optimization time can be further reduced to around 12.2ms by setting the Ceres Solver to evaluate the Jacobians with 4 threads.

V. CONCLUSION AND FUTURE WORK

In this paper, we propose a continuous-time VIO based on generalized Hermite splines, i.e., VICO. Specifically, we employ quintic Hermite spline in $\mathbb{R}^3$ for position and generalized cubic Hermite spline on S^3 for orientation to represent ego-motion, while the IMU acceleration and angular velocity measurements are synthesized according to this continuous-time parameterization scheme. VICO is further formulated as a sliding window based nonlinear optimization and the analytical form of the Jacobians is given with respect to the states. Experiments on TUM VI and EuRoC datasets shown that our system achieves state-of-the-art accuracy and real-time performance. For the future research, we intend to extend the system's capabilities to include online calibration and the integration of additional sensors. Moreover, we will also consider non-uniform splines to pursue a more adaptive continuous-time parameterization strategy.

REFERENCES

- [1] C. Cadena *et al.*, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, vol. 32, pp. 1309–1332, 2016.
- [2] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, pp. 1004–1020, 2017.
- [3] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Trans. Robot.*, vol. 33, pp. 1–21, 2015.
- [4] A. Haarbach, T. Birdal, and S. Ilic, "Survey of higher order rigid body motion interpolation methods for keyframe animation and continuous-time trajectory estimation," in *Proc. IEEE Int. Conf. 3D Vis.*, 2018, pp. 381–389.
- [5] T.-M. Nguyen, D. Duberg, P. Jensfelt, S. Yuan, and L. Xie, "SLICT: Multi-input multi-scale surfel-based lidar-inertial continuous-time odometry and mapping," *IEEE Robot. Autom. Lett.*, vol. 8, pp. 2102–2109, 2022.
- [6] X. Lang *et al.*, "Coco-LIC: Continuous-time tightly-coupled lidar-inertial-camera odometry using non-uniform B-spline," *IEEE Robot. Autom. Lett.*, vol. 8, pp. 7074–7081, 2023.
- [7] D. Hug, P. Banninger, I. Alzugaray, and M. Chli, "Continuous-time stereo-inertial odometry," *IEEE Robot. Autom. Lett.*, vol. 7, pp. 6455–6462, 2022.
- [8] J. Mo and J. Sattar, "Continuous-time spline visual-inertial odometry," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2022, pp. 9492–9498.
- [9] S. Lovegrove, A. Patron-Perez, and G. Sibley, "Spline fusion: A continuous-time representation for visual-inertial fusion with application to rolling shutter cameras," in *Proc. Brit. Mach. Vis. Conf.*, 2013.
- [10] S. R. Anderson, F. Dellaert, and T. D. Barfoot, "A hierarchical wavelet decomposition for continuous-time slam," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2014, pp. 373–380.
- [11] F. Dümbsen, C. T. Holmes, and T. D. Barfoot, "Safe and smooth: Certified continuous-time range-only localization," *IEEE Robot. Autom. Lett.*, vol. 8, pp. 1117–1124, 2022.
- [12] T. Y. Tang, D. J. Yoon, and T. D. Barfoot, "A white-noise-on-jerk motion prior for continuous-time trajectory estimation on $SE(3)$," *IEEE Robot. Autom. Lett.*, vol. 4, pp. 594–601, 2018.
- [13] J. Lv, X. Lang, J. Xu, M. Wang, Y. Liu, and X. Zuo, "Continuous-time fixed-lag smoothing for lidar-inertial-camera SLAM," *IEEE/ASME Trans. Mechatronics*, vol. 28, pp. 2259–2270, 2023.
- [14] M.-J. Kim, M.-S. Kim, and S. Y. Shin, "A general construction scheme for unit quaternion curves with simple high order derivatives," in *Proc. ACM SIGGRAPH Conf. Comput. Graph.*, 1995, pp. 369–376.
- [15] X. Lang, J. Lv, J. Huang, Y. Ma, Y. Liu, and X. Zuo, "Ctrl-VIO: Continuous-time visual-inertial odometry for rolling shutter cameras," *IEEE Robot. Autom. Lett.*, vol. 7, pp. 11 537–11 544, 2022.
- [16] E. Mueggler, G. Gallego, H. Rebecq, and D. Scaramuzza, "Continuous-time visual-inertial odometry for event cameras," *IEEE Trans. Robot.*, vol. 34, pp. 1425–1440, 2017.
- [17] H. Ovrén and P.-E. Forssén, "Trajectory representation and landmark projection for continuous-time structure from motion," *Int. J. Robot. Res.*, vol. 38, pp. 686–701, 2019.
- [18] C. Sommer, V. C. Usenko, D. Schubert, N. Demmel, and D. Cremers, "Efficient derivative computation for cumulative B-splines on Lie groups," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11 145–11 153.
- [19] J. C. Johnson, J. G. Mangelson, and R. W. Beard, "Continuous-time trajectory estimation for differentially flat systems," *IEEE Robot. Autom. Lett.*, vol. 8, pp. 145–151, 2023.
- [20] G. S. Chirikjian, *Stochastic Models, Information Theory, and Lie Groups, Volume 2: Analytic Methods and Modern Applications*, ser. Applied and Numerical Harmonic Analysis. Boston, USA: Birkhäuser Boston, 2012, p. 40.
- [21] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. Int. Joint Conf. Artif. Intell.*, 1981.
- [22] J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 1994, pp. 593–600.
- [23] S. Agarwal *et al.*, *Ceres Solver*. [Online]. Available: <https://github.com/ceres-solver/ceres-solver>
- [24] D. Schubert, T. Goll, N. Demmel, V. C. Usenko, J. Stückler, and D. Cremers, "The TUM VI benchmark for evaluating visual-inertial odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robot. Syst.*, 2018, pp. 1680–1687.
- [25] M. Burri *et al.*, "The EuRoC micro aerial vehicle datasets," *Int. J. Robot. Res.*, vol. 35, pp. 1157–1163, 2016.
- [26] M. Group, *evo: Python package for the evaluation of odometry and SLAM*, 2017. [Online]. Available: <https://github.com/MichaelGrupp/evo>
- [27] J. J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, pp. 611–625, 2016.